

Part 1 - Architecture Principles

1. Problem understanding and goals

The goal of this architecture is to support an English learning product where users learn vocabulary through music. The backend needs to store songs with lyrics, translations, word-level learning insights, and user-specific vocabulary state.

A user should be able to open a song, read the lyrics in the correct order, see translations, tap individual words, understand their meaning, and know whether each word is already known, still being learned, unknown, or ignored.

Because of this, the design should avoid storing all information in a single nested document. Instead, it should separate the original lyric structure used for display from the normalized word representation used for search and user-specific calculations.

The architecture should also keep global catalog data separate from user-specific data. Songs, lyrics, translations, and word insights are shared across users, but vocabulary status is specific to each user.

Assumptions

- The system receives songs with synchronized lyric lines from an external provider
- Tokenization, normalization, translation updating, and insight generation can be performed asynchronously after the song is imported.
- Translations may exist at line level, word level, or word-insight level.

Design principles

- Separation of concerns: display data, searchable data, translations, insights, and user state should be modeled separately.
- Explainable logic: difficulty scoring and vocabulary state calculations should be deterministic and easy to test.
- Extensibility: the model should support new languages, translation providers, and insight-generation strategies without requiring major schema changes.

- **User-state isolation:** user-specific learning progress should not duplicate or mutate global song catalog data.

2. High-level backend architecture

The proposed solution is a modular monolith with domain-oriented modules and clean architecture principles. Each module separates controllers, application services, domain logic, and persistence concerns. This keeps the system simple to deploy while remaining maintainable and extractable into independent services later if needed.

A modular monolith was chosen over a microservices architecture because the system is early-stage and domain boundaries are still being validated. Extracting modules into independent services prematurely would add deployment and operational complexity without a clear benefit. The tradeoff is that all modules share the same deployment unit, if one module has a performance issue it affects the whole system. The modular structure mitigates this by keeping domain logic isolated, making extraction straightforward if a module needs to scale independently later.

The system is organized around two types of data:

- **Global catalog data:** songs, lyrics, word occurrences, word insights, and embedded translations. Shared across all users and never mutated by user activity.
- **User-specific learning data:** vocabulary status, attempt history, and user-specific song difficulty. Isolated per user and never stored inside catalog documents.

The most important architectural decision is that user state never lives inside catalog data. The same song can be experienced differently by any number of users without duplicating or mutating shared documents.

The chosen collections will be explained in section 3.

Main backend modules

Song Catalog Module

Responsible for storing song and artist metadata.

It owns entities such as:

- Song

- Artist

This module does not store user-specific learning state.

Lyrics Module

Responsible for storing the original lyric structure used for display.

It owns entities such as:

- LyricLine

The original lyric text is preserved exactly as received, including line order and optional timing information. This allows the app to display synchronized lyrics without depending on tokenized or normalized word data.

Translations are stored together with their parent entity. They are always read alongside the line they belong to, so keeping them separate would add latency with no benefit.

Lyrics Processing Module

Responsible for transforming lyric lines into searchable word-level data.

It handles:

- tokenization;
- normalization;
- word position extraction;
- word occurrence creation.

It owns entities such as:

- WordOccurrence

This module produces the indexable representation of the lyrics but does not replace the original lyrics display model. It runs asynchronously after a song is imported and never modifies the original lyric data.

Word Insight Module

Responsible for storing learning metadata about normalized words.

It owns entities such as:

- WordInsight

A word insight may include:

- normalized word;
- source language;
- translations;
- difficulty;
- frequency;
- examples;
- song references;
- image references;
- source metadata.

Word insights are global. They describe the word itself, not whether a specific user knows it. Translations are stored together with the insight they belong to, following the same reasoning as the Lyrics Module.

User Vocabulary Module

Responsible for storing the relationship between a user and a word.

It owns entities such as:

- UserVocabulary

It tracks:

- status: `unknown`, `learning`, `known`, or `ignored` ;
- correct attempt count;
- incorrect attempt count;
- last practiced date.

Song Insight Query Module

Responsible for building user-specific song insights on demand.

This module does not own persistent entities. It composes data from existing entities across modules to calculate values such as:

- total words;
- unique words;
- known and unknown word counts;

- repeated words;
- difficulty score for a specific user.

Because this query crosses multiple modules, results can be cached at the application layer when the underlying data has not changed.

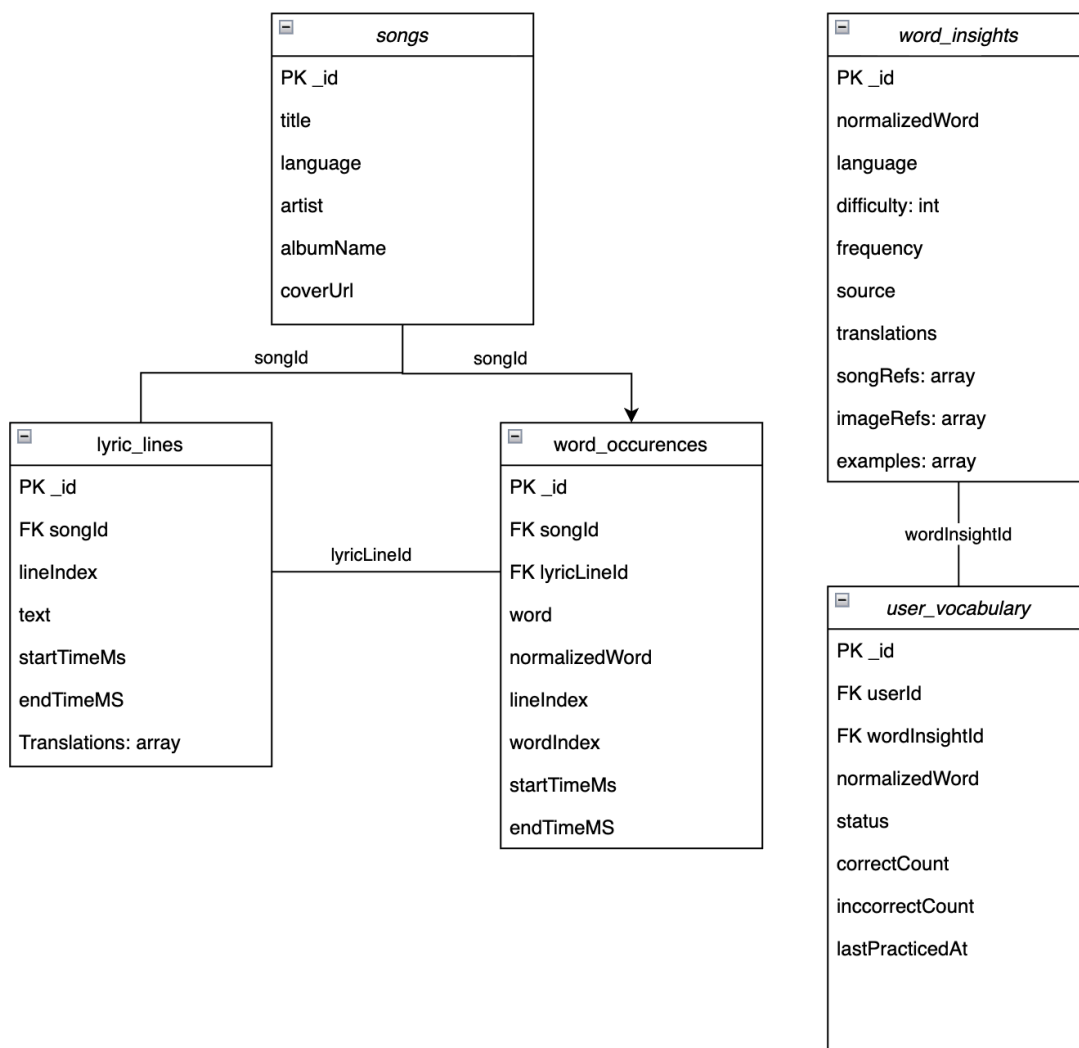
Asynchronous execution

Asynchronous behavior is owned by the module that defines the use case.

A shared background execution mechanism can be used to run these use cases without blocking user-facing requests.

Each domain module owns its async use cases, while a shared job execution mechanism handles retries, scheduling, and failure management.

Sección 3: Data Model



Technology choice

The data model is implemented using MongoDB. The decision is based on three factors:

- **Access patterns are predictable:** The most frequent queries always follow the same shape: lines by song, occurrences by song and word, vocabulary by user. There are no complex joins that would benefit from a relational engine.
- **Data is naturally hierarchical.** Translations, examples, and song references are always read together with their parent entity. MongoDB's document model allows embedding these directly, avoiding unnecessary roundtrips.
- **The schema needs flexibility.** Supporting multiple languages, translation providers, and insight types without requiring migrations is a meaningful advantage during early product development.

Overview

The data model is divided into two clearly separated layers:

- **Global catalog data:** shared across all users: `Song`, `Artist`, `LyricLine`, `WordOccurrence`, `Translation`, `WordInsight`
- **User-specific data:** isolated per user: `UserVocabulary`

No user state is stored inside catalog entities. The same song can be experienced differently by any number of users without duplicating or mutating shared data.

3.1 songs

The artist is stored as an embedded subdocument. This is appropriate given that artist data is always read together with the song and remains small in size. If artist-level queries become necessary in the future, the artist can be extracted to its own collection without breaking the rest of the model.

```
{
  _id: ObjectId,
  title: "Perfect",
  language: "en",
  albumName: "albumName",
  coverUrl: "https://..."
```

```

    artist: {
      id: "artist_001",
      name: "Ed Sheeran",
      imageUrl: "https://..."
    }
  }
}

```

3.2 lyric_lines

Stored as a separate collection from `songs` because lines are queried individually by `lineIndex`, by word search, and their volume per song is variable. Translations are embedded as an array because they are always read together with the line and have bounded size.

```

{
  _id: ObjectId,
  songId: "song_001",
  lineIndex: 0,
  text: "I found a love for me",
  startTimeMs: 1200,
  endTimeMs: 4200,
  translations: [
    { language: "es", text: "Encontré un amor para mí", provider: "manual" },
    { language: "pt", text: "Encontrei um amor para mim", provider: "deepl" }
  ]
}

```

```
db.lyric_lines.createIndex({ songId: 1, lineIndex: 1 })
```

3.3 word_occurrences

Generated asynchronously after a song is imported. Each document represents a single word token at a specific position. This collection enables searching a word within a song, finding all its occurrences, and calculating song insights.

`songId` is stored directly on each document as an intentional denormalization. The most frequent query — "give me all occurrences of this word in this song"

would otherwise require a two-step lookup through `lyric_lines`. Since the catalog is immutable after import, the cost of keeping this field in sync is negligible.

```
{
  _id: ObjectId,
  songId: "song_001",
  lyricLineId: "line_001",
  word: "found",
  normalizedWord: "find",
  lineIndex: 0,
  wordIndex: 1,
  startTimeMs: 1200,
  endTimeMs: 4200
}
```

```
db.word_occurrences.createIndex({ songId: 1, normalizedWord: 1 }) // find all occurrences of a word within a specific song
db.word_occurrences.createIndex({ normalizedWord: 1 }) // find all occurrences of a word across all songs
db.word_occurrences.createIndex({ songId: 1, lineIndex: 1 }) // find all words in a specific line of a song
```

3.4 `word_insights`

Global vocabulary metadata. Translations, examples, and `imageRefs` are embedded because they are always read together with the insight. `songRefs` is also embedded as a lightweight array — it only stores `songId`, title, and occurrence count, without requiring full song data.

```
{
  _id: ObjectId,
  normalizedWord: "darling",
  language: "en",
  difficulty: 2,
  frequency: 12,
  source: "song",

```

```

translations: [
  { language: "es", text: "cariño" }
],
songRefs: [
  { songId: "song_001", title: "Perfect", occurrences: 3
}
],
imageRefs: [
  { id: "img_001", url: "https://example.com/darling.png"
}
],
examples: [
  {
    text: "Darling, just dive right in",
    translations: [
      { language: "es", text: "Cariño, simplemente lánzat
e" }
    ]
  }
]
]
}

```

```

db.word_insights.createIndex({ normalizedWord: 1, language:
1 }, { unique: true })
db.word_insights.createIndex({ language: 1, difficulty: 1
})
db.word_insights.createIndex({ language: 1, frequency: -1
})

```

3.5 `user_vocabulary`

The only collection containing user-specific data. References `word_insights` by `wordInsightId` but never duplicates its content. `normalizedWord` is denormalized here to avoid a lookup to `word_insights` on the most frequent query: "does this user already know this word?", which is called for every word in a song when calculating the difficulty score.

```

{
  _id: ObjectId,

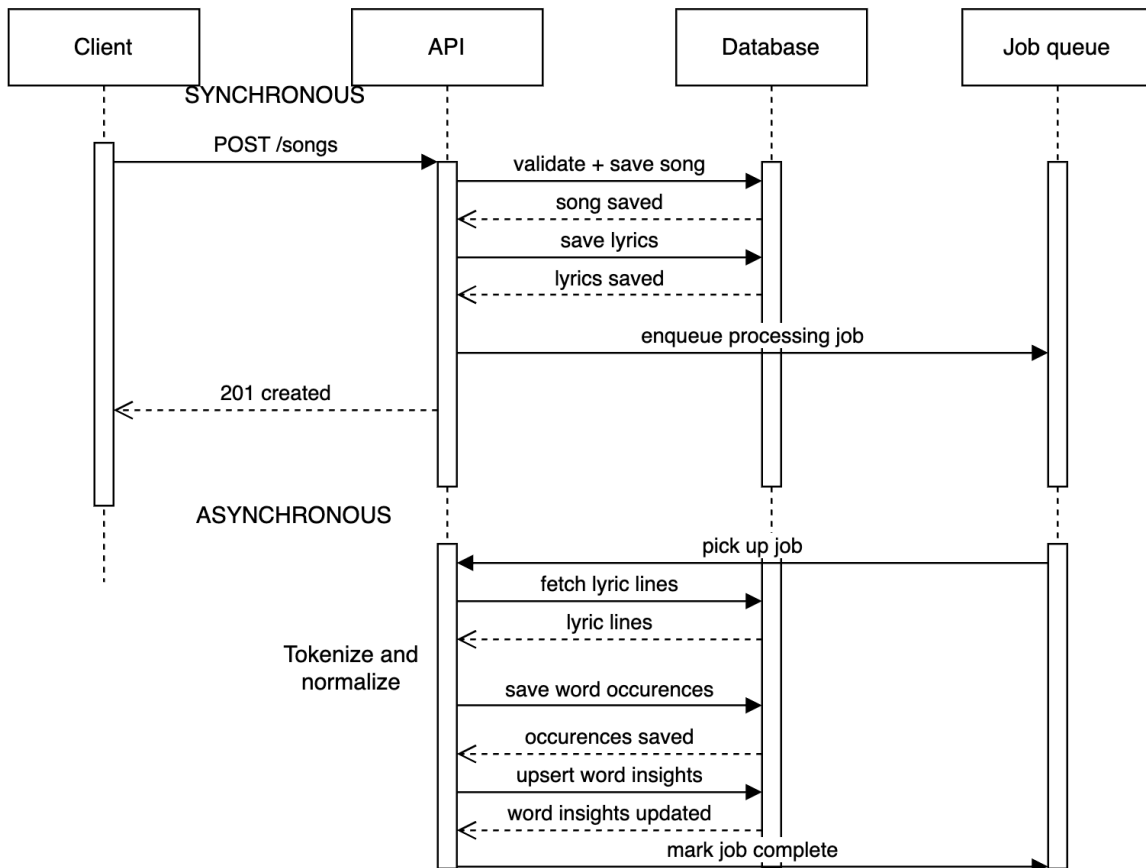
```

```
userId: "user_001",
wordInsightId: "insight_001",
normalizedWord: "darling",
status: "learning",
correctAttemptCount: 3,
incorrectAttemptCount: 1,
lastPracticedAt: ISODate("2025-05-10T14:00:00Z")
}
```

```
db.user_vocabulary.createIndex({ userId: 1, status: 1 })
db.user_vocabulary.createIndex({ userId: 1, wordInsightId:
1 }, { unique: true })
db.user_vocabulary.createIndex({ userId: 1, normalizedWord:
1 })
db.user_vocabulary.createIndex({ userId: 1, lastPracticedA
t: -1 })
```

4. Lyric import and processing flow

When a song is imported, the system separates the work into two phases: a synchronous phase that persists the minimum data needed to respond quickly, and an asynchronous phase that enriches the catalog without blocking the user-facing request.



4.1 Synchronous phase

Executed immediately on import request:

- Validate input: the incoming body is validated.
- Persist song and lyric lines: the **Song** document is created or upserted, with the **artist** embedded as a subdocument. Each lyric line is stored as a separate **LyricLine** document, preserving the original text exactly as received, including optional timing data and line-level translations.
- Return response: a **201** response is returned immediately. The song is now available for display. Processing is not yet complete, but the original lyrics are ready.

4.2 Asynchronous phase

Triggered after the synchronous phase by enqueueing a background job:

- Tokenize lines: each lyric line is split into individual word tokens. Punctuation is stripped, and casing is normalized.

- Normalize words: each word is normalized to its base form. For example, "found" → "find". This is the `normalizedWord`.
- Create word occurrences: one `WordOccurrence` document is created per word, storing the original word, normalized word, `lineIndex`, `wordIndex`, and timing data inherited from the parent line.
- Upsert word insights: for each unique `normalizedWord`, the corresponding `WordInsight` is updated: frequency is incremented and the song is added to `songRefs` if not already present.
- Mark job complete: the background job is marked as done. If any step fails, the job is retried by the shared execution mechanism.

4.3 Assumptions

- Normalization of words is handled by a third-party, appropriate for the song's language. The system passes the `language` field from the `Song` document to select the correct strategy.
- If a word cannot be normalized (unknown token, unsupported language), the `normalizedWord` falls back to the lowercase surface form.
- The asynchronous phase is idempotent re-running it on the same song produces the same result, making retries safe.

5. Translation resolution

Translations are embedded as arrays within their parent entity, either a `LyricLine` or a `WordInsight`. Each entry contains a `language` and a `text` field.

When a translation is requested, the system applies the following order:

- **Exact match:** the translation for the requested language is returned.
- **No translation:** If no translation for the requested language is found, for learning flows, returning an unrelated language may confuse the user. Therefore, the safer default is to return null with `translationStatus`: "missing".

A fallback translation can be returned only if the client explicitly allows fallback behavior.

Translation updating is always asynchronous, no external provider is called during a user-facing request. The `provider` field ("manual" , "deepl" , "google")

allows the system to prioritize manual translations over auto-generated ones when both exist for the same language.

Supporting a new target language does not require schema changes, a background job adds entries to existing arrays without modifying existing ones.

6. Core query patterns

6.1 Word occurrence query

Given a song and a word, return return line position, word position, original line text, normalized word, and optimal timing data.

Input: `songId`, `normalizedWord`

Requires a `$lookup` to bring in the original line text from `lyric_lines`:

Query:

```
db.word_occurrences.aggregate([
  { $match: { songId: "song_001", normalizedWord: "find" } },
  { $lookup: {
    from: "lyric_lines",
    localField: "lyricLineId",
    foreignField: "_id",
    as: "line"
  } },
  { $project: {
    word: 1,
    normalizedWord: 1,
    lineIndex: 1,
    wordIndex: 1,
    startTimeMs: 1,
    endTimeMs: 1,
    lineText: { $arrayElemAt: ["$line.text", 0] }
  } }
])
```

Output:

```
[
  {
    word: "found",
    normalizedWord: "find",
    lineIndex: 0,
    wordIndex: 1,
    lineText: "I found a love for me",
    startTimeMs: 1200,
    endTimeMs: 4200
  }
]
```

6.2 Line and word translation query

return available translations for a lyric line or specific word occurrence, and explain how missing translations are represented

In both cases the application filters the embedded `translations` array by the requested language and returns.

For a lyric line:

Query:

```
db.lyric_lines.findOne({ _id: "line_001" })
```

Output:

```
{
  language: "es",
  text: "Encontré un amor para mí"
}
```

For a word occurrence:

Query:

```
db.word_insights.findOne({ normalizedWord: "find", language: "en" })
```

Output:

```
{  
  language: "es",  
  text: "encontrar"  
}
```

If the requested language is not found:

```
{  
  "translation": null,  
  "translationStatus": "missing"  
}
```

Optionally, if the client requests fallback behavior, the first available translation can be returned with `isFallback: true`.

6.3 Song insights query

Given a song and user, return total words, unique words, known/unknown words, repeated words, relevant user vocabulary status, and a `difficultyScore`

`word_occurrences` and `user_vocabulary` are queried independently and crossed at the application layer, since they live in separate domains (catalog vs. user data) and the cross-reference is by `normalizedWord`, which is denormalized on both collections precisely for this purpose:

```
const uniqueWords = db.word_occurrences.distinct("normalizedWord", {  
  songId: "song_001"  
})
```

```
const userVocabulary = db.user_vocabulary.find({  
  userId: "user_001",
```

```
normalizedWord: { $in: uniqueWords }  
}))
```

The application classifies each unique word by status. Words with no `user_vocabulary` record are treated as `unknown` by default.

```
{  
  songId: "song_001",  
  userId: "user_001",  
  totalWords: 120,  
  uniqueWords: 60,  
  knownWords: 40,  
  unknownWords: 15,  
  learningWords: 4,  
  ignoredWords: 1,  
  repeatedWordOccurrences: 60,  
  vocabularyStatus: [  
    {  
      normalizedWord: "love",  
      status: "known"  
    },  
    {  
      normalizedWord: "find",  
      status: "learning"  
    },  
    {  
      normalizedWord: "darling",  
      status: "unknown"  
    }  
  ],  
  difficultyScore: 0.25  
}
```

Difficulty score formula:

```
difficultyScore = unknownUniqueWords / totalUniqueWords
```

In this first version, only `unknown` words are counted as difficult. Words marked as `learning` are returned separately, but they do not increase the difficulty score. This keeps the formula simple, deterministic, and easy to test.

A richer version could count `learning` words as partially difficult:

```
difficultyScore = (unknownUniqueWords + learningUniqueWords * 0.5) / totalUniqueWords
```

7. Scalability considerations

More songs and lyrics: `word_occurrences` is the collection that grows fastest. The indexes defined in Section 3 keep queries fast regardless of volume, and the asynchronous processing pipeline absorbs the extra load without affecting user-facing requests.

More users: `user_vocabulary` grows linearly, at most one document per word per user. The song insights query is the most expensive per-user operation and can be cached at the application layer, discarded when the user's vocabulary state changes.

More languages: No schema changes are required. Translations are embedded arrays, so a background job can add new language entries to existing documents. Supporting a new source language only requires adding a normalization strategy to the Lyrics Processing Module.

8. Error handling and invalid input

The backend should validate imported songs before storing them.

Validation examples:

- Missing song title.
- Missing or unsupported language.
- Empty lyrics array.
- Duplicated `lineIndex` values.
- Negative timestamps.
- `endTimeMs` lower than `startTimeMs`.
- Invalid translation language.

- Invalid or empty lyric text.

For synchronous import errors: the API should return a 400 response with a clear validation message.

For asynchronous processing errors: the original song and lyric lines should remain stored, but the processing job should be marked as failed or retried. A failed processing job should not corrupt the original catalog data.

If tokenization or normalization fails for a specific word, the system can fallback to the lowercase surface form. If the whole song processing fails, the song can remain available for display while word search and insights remain unavailable until processing succeeds.

Missing translations should not be treated as server errors. They should be represented explicitly as null or as a missing translation state.